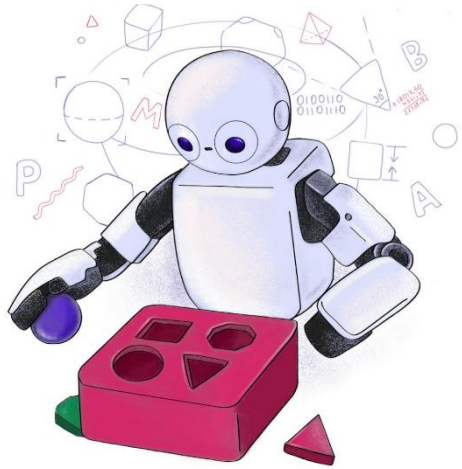
TP557 - Tópicos avançados em IoT e Machine Learning:

Regressão com DNNs (Parte I)



Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

O que vamos ver?

- Anteriormente, vimos como minimizar iterativamente a função de erro usando o gradiente descendente.
  - Inicializa-se os pesos, em geral, com valores aleatórios;
  - Mede-se o erro causado pelo valor atual dos pesos;
  - Usa-se a informação obtida através do erro (i.e., vetor gradiente) para atualizar os pesos;
  - Repete-se o processo até a convergência ou até que um critério de parada seja atingido.
- Em termos gerais, é assim que as redes neurais são treinadas.
- Portanto, neste tópico, nós veremos como criar uma rede neural que atinja o mesmo objetivo do exemplo anterior, encontrar uma ***função linear*** que aproxime um conjunto de dados.

Conjunto de dados de treinamento

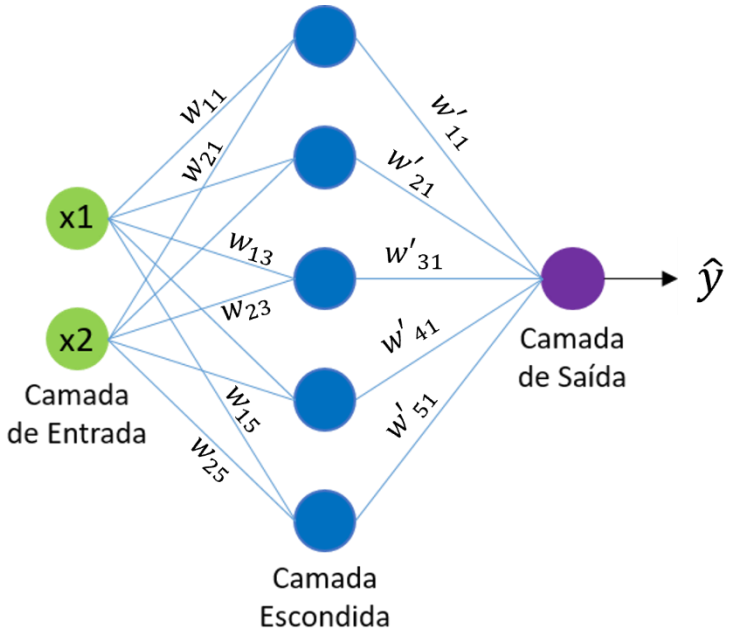
$$\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$$
$$\boldsymbol{y} = \{-3, -1, 1, 3, 5, 7\}$$

- Ao lado temos o conjunto de dados que usamos anteriormente.
- Nosso ***objetivo*** é encontrar um ***modelo que mapeie*** os valores de $\boldsymbol{x}$ em $\boldsymbol{y}$ de forma ótima (no sentido da minimização do erro).
- Antes, nós usamos o ***gradiente descendente*** para ***otimizar os pesos de uma função hipótese com formato de reta***.
- Agora, treinaremos uma rede neural para resolver o problema do mapeamento.

O modelo

$$\hat{y}(n) = a_0 + a_1 x_1(n) + \cdots + a_K x_K(n)$$

$$\hat{y}(n) = a_0 + a_1 x_1(n) + a_2 x_1^2(n) + \cdots + a_K x_1^K(n)$$



- Notem que independentemente se estamos usando ***hiperplanos***, ***polinômios*** ou ***redes neurais***, em todos os casos temos uma ***função*** (i.e., um ***modelo***) e o nosso ***objetivo é encontrar seus pesos de forma que o erro seja minimizado***.
- Em outras palavras, queremos encontrar os pesos que façam com que a função mapeie $\boldsymbol{x}$ em $\boldsymbol{y}$ com o menor erro possível.

O código da rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Usaremos as APIs da biblioteca TensorFlow para ***criar***, ***treinar*** e ***avaliar*** nossas redes neurais.
  - ***Application programming interface*** (**API**): conjunto de regras (e.g., funções, classes, etc.) que um software oferece para que desenvolvedores ou outros programas possam interagir com ele.

Definindo o conjunto de treinamento

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- As primeiras duas linhas definem o conjunto de treinamento.
- Ou seja, os valores de x e y que usaremos para otimizar o modelo durante as iterações e épocas de treinamento.
- Cada valor de x corresponde a um valor de y.
- O Tensorflow espera que o conjunto de dados sejam *arrays* NumPy.

Definindo uma rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

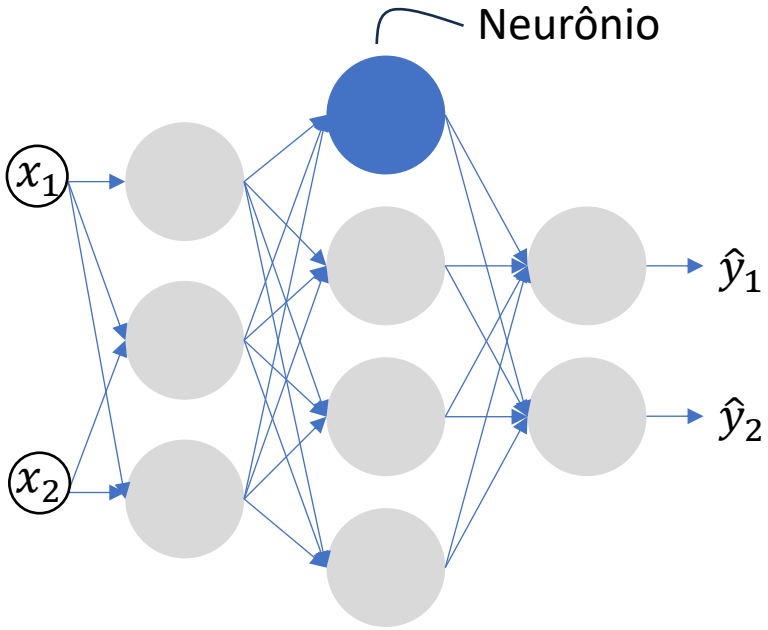
model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

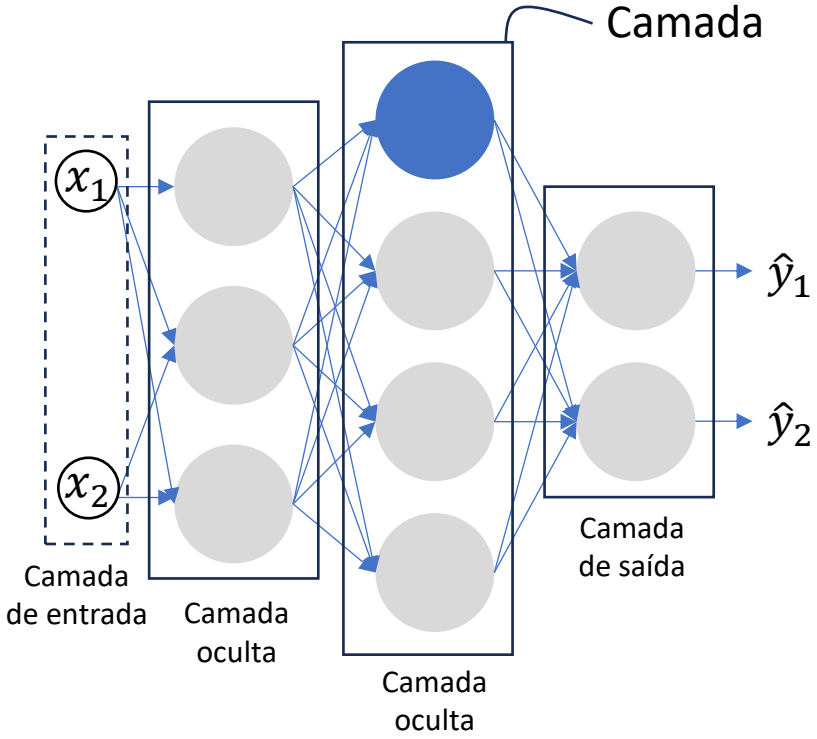
- Na sequência, temos a definição da rede neural.
- É uma rede neural muito simples, uma das mais simples que veremos.
- Antes de discutirmos o código, vamos relembrar alguns termos para que possamos entendê-lo.

Relembrando alguns termos



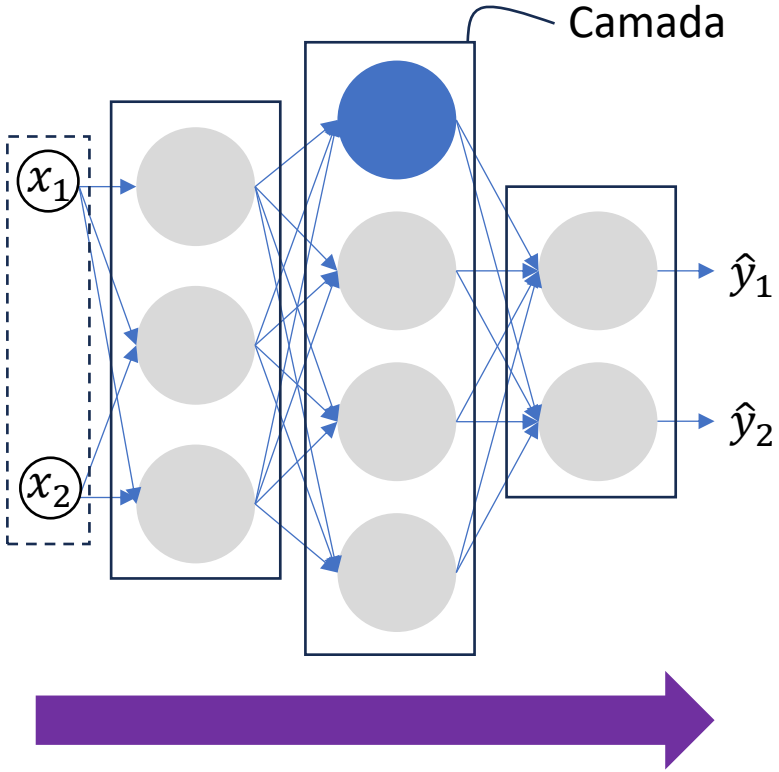
- Na rede neural ao lado, cada um dos círculos é um neurônio ou nó.

Relembrando alguns termos



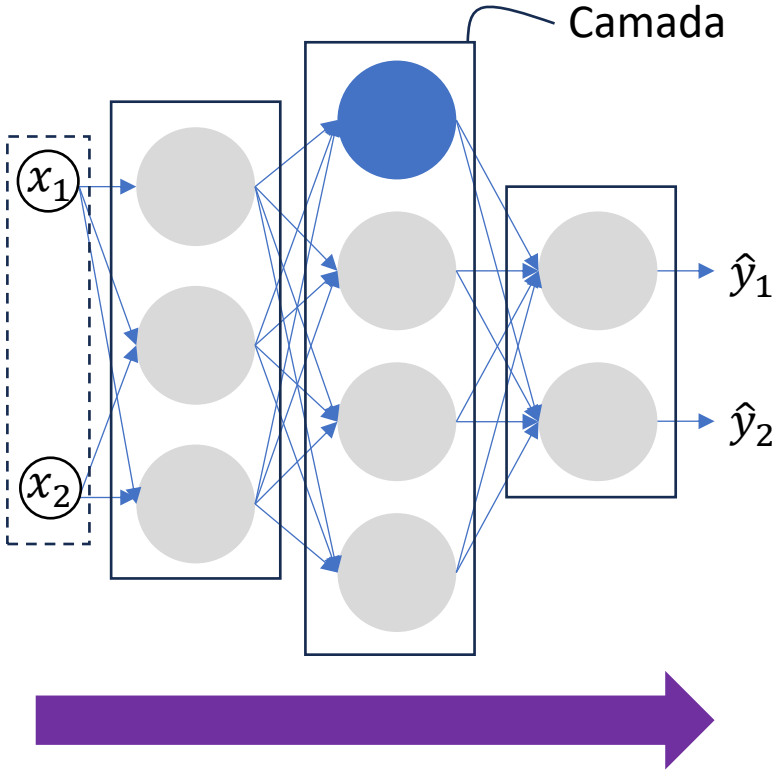
- Nós chamamos cada um dos ***conjuntos de neurônios*** em um retângulo de ***camada***.
- A rede ao lado tem ***duas camadas ocultas*** e ***uma camada de saída***.
- **OBS**.: alguns autores chamam o ponto de transferência dos atributos para a rede de ***camada de entrada*** (retângulo com linha tracejada).
  - Essa camada ***não tem pesos***.
  - Ela normalmente é usada para aplicar ***transformações de dimensão*** aos dados de entrada.

Relembrando alguns termos



- Em redes de ***alimentação direta***, as camadas estão conectadas em ***sequência***.
- As ***informações fluem em uma única direção***, ou seja, elas fluem da entrada passando pelas camadas ocultas e, finalmente, para a camada de saída, ***sem recursões***.

Relembrando alguns termos



- Na terminologia do Tensorflow, nós usamos o termo ***sequential*** para definir este tipo de rede.
- Além disso, vemos que as ***saídas de uma camada estão conectadas a todos os neurônios da próxima camada***, criando conexões ***densas***.
- Usaremos o termo ***densas*** para definir estes tipos de camadas da rede.

Definindo uma rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Voltando ao código, nós vemos o termo ***sequential*** e, portanto, temos a definição de uma rede de ***alimentação direta ou sequencial***.
- Dentro dos colchetes, ***listamos as camadas*** dessa rede neural.
- Nesse exemplo, a ***lista*** contém apenas um elemento, portanto, temos apenas uma ***camada***.

Definindo uma rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Essa camada é do tipo ***densa*** (classe *Dense*), então sabemos que é uma camada ***densamente conectada***.
- O parâmetro ***units*** define quantos neurônios a camada possui.
- Podemos ver que essa camada tem ***apenas um neurônio***.
- Portanto, esse código define a rede neural mais simples possível.
  - Há apenas uma camada com um único neurônio nela.

Definindo uma rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Um parâmetro que definimos ***apenas para a primeira camada*** (neste caso a única) de uma rede neural é o formato (i.e., ***dimensões***) das ***entradas***.
- No exemplo, o parâmetro ***input_shape*** tem o valor 1, que indica a dimensão da entrada, ou seja, a ***quantidade de atributos de entrada*** do neurônio.
- Isso significa que o neurônio tem apenas uma entrada, o atributo x.

Definindo uma rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Quando vimos o funcionamento dos neurônios, aprendemos que eles possuem uma ***função de ativação***, que tem como entrada a ***combinação ponderada das entradas pelos pesos sinápticos mais o peso de bias*** e que faz um ***mapeamento não linear*** de sua entrada na saída.
- Porém, como queremos encontrar um ***mapeamento linear entre a entrada e a saída***, ***não usaremos*** nenhuma ***função de ativação***.

Definindo uma rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

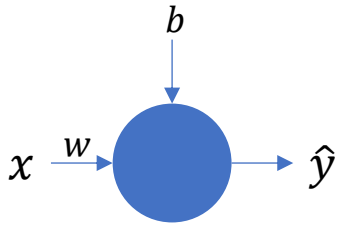
model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Lembrem-se nosso objetivo é encontrar os pesos de uma função hipótese, do tipo $\hat{y} = b + wx$, ***modelada pela rede neural***.
- A função de ativação pode ser definida através do parâmetro ***activation*** da classe ***Dense***.
- Por padrão, a ativação é definida como ***None***, ou seja, não se tem ativação.

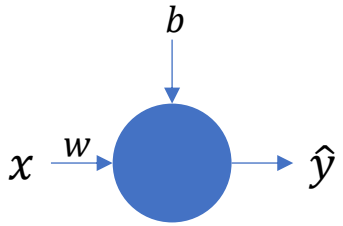
Nossa rede neural



$$\hat{y} = b + wx$$

- Visualmente, nossa rede neural se parece com a figura ao lado.
- Nós temos apenas ***uma camada*** com um ***único neurônio*** nela.
- O neurônio tem apenas uma entrada, o atributo, x.
  - Por isso a dimensão da entrada é igual a 1.

Nossa rede neural



$$\hat{y} = b + wx$$

- Nenhuma função de ativação é definida.
  - Isso se deve ao fato de queremos encontrar um ***mapeamento linear***.
- Ele irá aprender os pesos (w e b) que mapeiam x em y da ***melhor forma possível***, baseado no ***conjunto de treinamento***.
- Esse modelo é conhecido na literatura como ***Perceptron***.

Compilando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Depois de termos definido o modelo, precisamos compilá-lo.
- Ao compilarmos o modelo, devemos definir a ***função de erro (loss)*** e um ***otimizador***.
- Como antes, usaremos o ***erro quadrático médio*** como função de erro.
- **Obs**.: O Tensorflow se encarrega de realizar os cálculos de erro, portanto, não precisamos nos preocupar com isso.

Compilando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- O ***otimizador*** é o SGD, que significa *Stochastic Gradient Descent*.
- Na verdade, ele implementa o ***gradiente descendente em mini-lotes***.
- Ele segue o processo que vimos anteriormente, o qual calcula o vetor gradiente usando um subconjunto de amostras do conjunto de treinamento.
- O SGD usa o vetor gradiente para caminhar pela (descer a) superfície de erro até atingir ***um*** mínimo.

Treinando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- O treinamento propriamente dito é feito através do método *fit()*.
- O que ele faz é atualizar ***iterativamente*** os pesos do modelo usando o SGD.
- O treinamento é feito por 500 épocas.
- Uma ***época*** representa ***uma passagem completa pelo conjunto de treinamento*** durante o treinamento do modelo.

Treinando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Uma ***iteração*** representa ***uma atualização dos pesos***.
- Assim, uma ***época é composta por um conjunto de iterações***.
- Por exemplo, se o conjunto de treinamento tem 1000 exemplos e está sendo treinado em *mini-batches* de 100 exemplos, então uma época é composta por 10 iterações, i.e., os ***pesos são atualizados 10 vezes***.
- Por padrão, o tamanho do *mini-batch* é igual a 32.

Treinando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- O tamanho do *mini-batch* é definido através do parâmetro ***batch_size*** do método *fit()*.
- Porém, como no código acima o número de exemplos de treinamento é menor do que o tamanho padrão, ***1 iteração se torna igual a 1 época***.
- Ou seja, todos os exemplos são usados para se atualizar os pesos → **GDB**.

Treinando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Cada ***iteração*** de atualização executa as etapas que discutimos antes:
  - dá-se um palpite a respeito dos valores dos pesos (inicialmente, eles são aleatórios),
  - mede-se o erro causado por esse palpite,
  - atualiza-se os pesos usando-se o gradiente do erro,
  - e repete-se o processo, aqui, até se completar as 500 épocas.

Treinando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Percebam que não estamos fazendo, nem faremos, nenhum cálculo nós mesmos (i.e., cálculo do erro, vetor gradiente, atualizações).
- Nós deixaremos o TensorFlow fazer isso por nós.
- Essa é uma das grandes vantagens do TensorFlow, principalmente quando temos redes neurais profundas.

Treinando a rede neural

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Ao final do treinamento, o método *fit()* retorna um objeto do tipo *History*.
- Ele contém, além de outros atributos, um ***dicionário*** chamado de ***history*** contendo o ***erro*** e ***todas as métricas extras*** medidas ***ao final de cada época*** no conjunto de treinamento e no conjunto de validação (se houver).

Realizando predições

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

- Depois de treinado, podemos usar o modelo para predizer o valor de y para um determinado x.
- A predição também é chamada de ***inferência***.
- Predições são feitas com o método *predict()*.

Realizando predições

```
x = np.array([-1.0,  0.0, 1.0, 2.0, 3.0, 4.0])
y = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0])

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')

history = model.fit(x, y, epochs=500)

print(model.predict([10.0]))
```

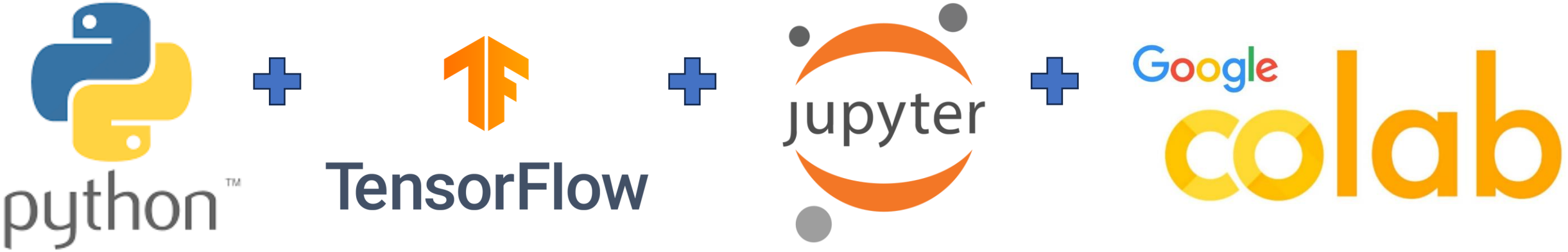
- Qual valor seria predito e impresso para $x = 10$?
- Sabemos das aulas anteriores que a melhor função hipótese é dada por

$$\hat{y} = -1 + 2x.$$

- Assim, quando $x = 10$, o valor de $\hat{y}$ deveria ser 19. Correto?
- Vamos ver no próximo exemplo, que, surpreendentemente, a resposta é não.

Exemplo

- Regressão com DNNs



Atividades

- Exercício #1: Regressão com DNNs (Parte I).
- Exercício #2: Regressão com DNN e dados ruidosos

Usando *callbacks*



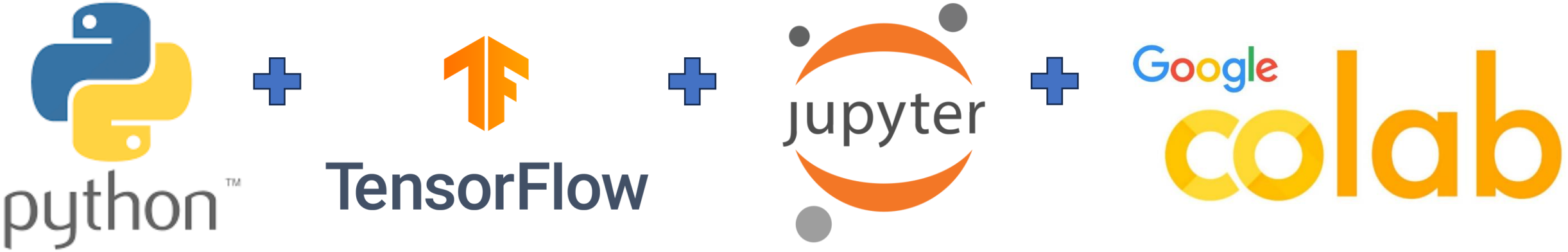
- E se o treinamento durar várias horas ou dias?
  - Isso é bastante comum, especialmente ao treinar grandes modelos com grandes conjuntos de dados.
- Nesse caso, não devemos salvar o modelo apenas ao final do treinamento, mas também ***salvar pontos de verificação em intervalos regulares*** durante o treinamento.

Usando *callbacks*

- Podemos configurar o método ***fit()*** para salvar pontos de verificação, mas como fazer esta configuração?
  - A resposta é: através de ***callbacks***.
  - ***callbacks*** são funções passadas como argumento para outras funções e chamadas quando um evento ocorrer.
- O método ***fit()*** possui um parâmetro chamado ***callbacks***, que permite especificar uma lista de ***callbacks*** que o modelo ***chamará durante o treinamento*** no ***início e no final do treinamento***, ***no início e no final de cada época*** e até ***antes e depois do processamento de cada mini-batch***.
- Veja todas as ***callbacks*** disponíveis em https://keras.io/callbacks/

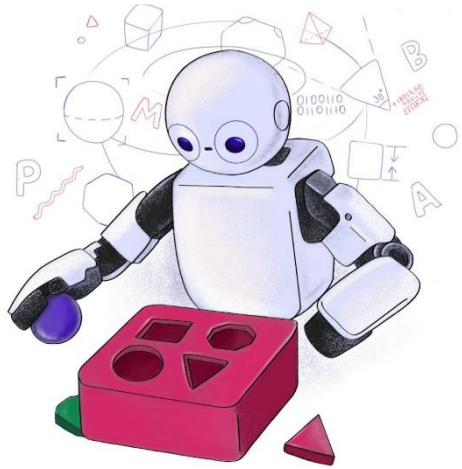
Exemplo

- Usando_callbacks.ipynb



TP557 - Tópicos avançados em IoT e Machine Learning:

Regressão com DNNs (Parte II)



Inatel

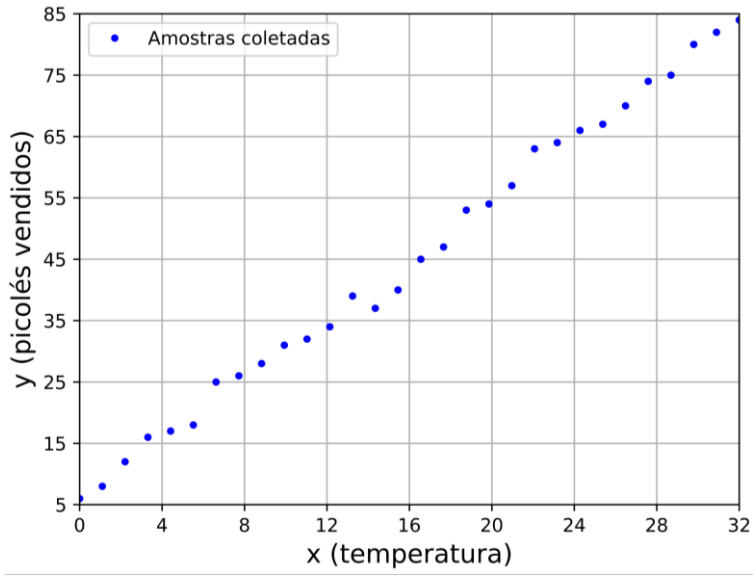
Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

O que vamos ver?

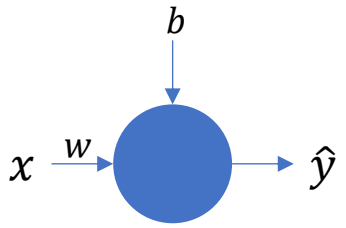
- Anteriormente, vimos através de um ***exemplo simples*** como usar a ***biblioteca TensorFlow*** para ***criar uma rede neural e resolver um problema de regressão***, ou seja, um problema de ajuste de curva.
- O objetivo era ter um ***contato inicial com a biblioteca*** e seus ***princípios básicos*** de funcionamento.
- Nesse tópico, vamos estender o que vimos anteriormente para um ***problema mais prático de regressão*** usando uma ***base de dados do mundo real***.

Regressão

$$\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$$
$$\boldsymbol{y} = \{-3, -1, 1, 3, 5, 7\}$$



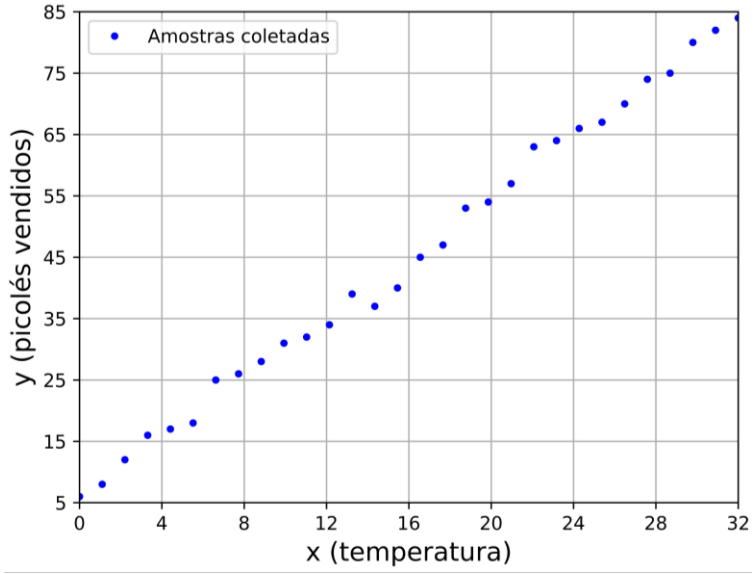
- Anteriormente, nós resolvemos um problema de regressão bem simples, onde queríamos mapear um único valor de x em um valor de saída, y.
- Fizemos o mapeamento usando uma reta como nossa função hipótese.



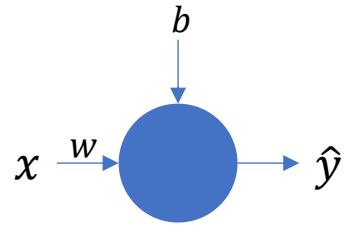
$$\hat{y} = b + wx$$

Regressão

$$\boldsymbol{x} = \{-1, 0, 1, 2, 3, 4\}$$
$$\boldsymbol{y} = \{-3, -1, 1, 3, 5, 7\}$$



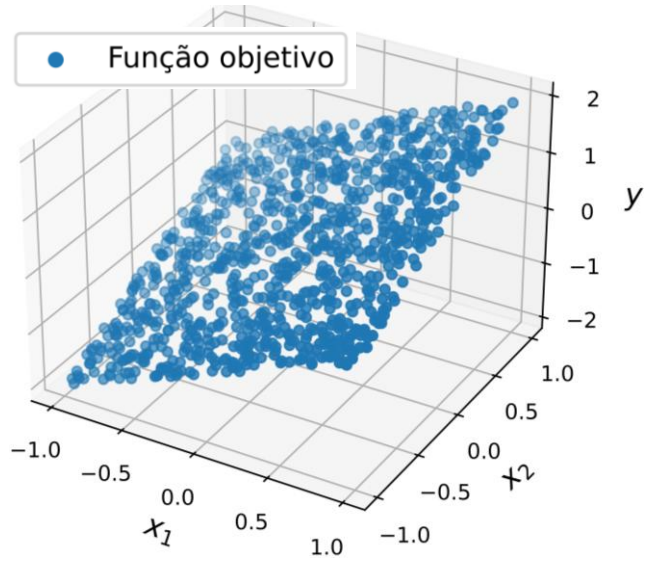
- Podemos fazer uma analogia com o problema de predizer o número de picolés que serão vendidos em um dia, y, dado a temperatura média daquele dia, x.
- Esse problema só tem um ***atributo*** de entrada, a temperatura, x.



$$\hat{y} = b + wx$$

Regressão

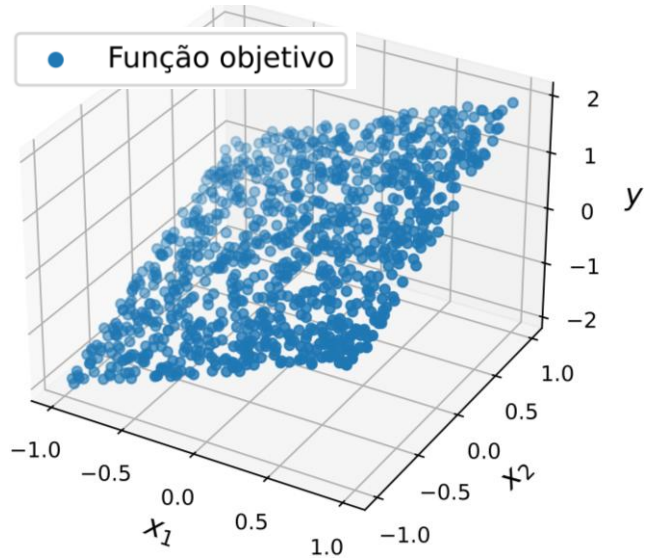
$$\boldsymbol{x}_1 = \{-1, 0, 1, 2, 3, 4\}$$
$$\boldsymbol{x}_2 = \{-8, 1, 3, 7, 0, 2\}$$
$$\boldsymbol{y} = \{-8, 0, 7, 1, 2, 3\}$$



- Mas e se quisermos um modelo que ***leve em consideração não só a temperatura***, mas o ***mês do ano também***?
- O modelo agora terá 2 ***atributos*** (i.e., 2 entradas).
- Por exemplo, a figura ao lado mostra a distribuição dos dados para um problema com dois atributos.
- Vejam que eles formam um plano.
- Portanto, existe uma relação linear entre os atributos e a saída.

Regressão

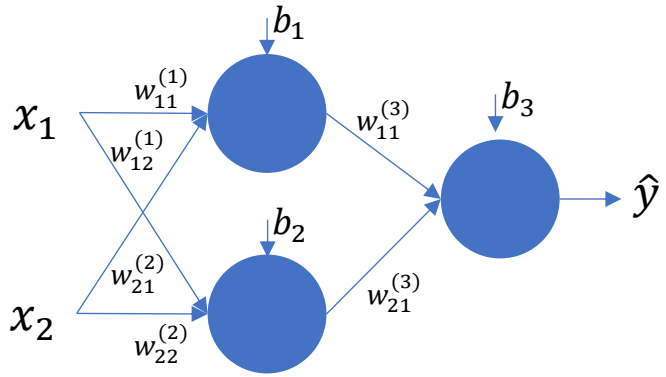
$$\boldsymbol{x}_1 = \{-1, 0, 1, 2, 3, 4\}$$
$$\boldsymbol{x}_2 = \{-8, 1, 3, 7, 0, 2\}$$
$$\boldsymbol{y} = \{-8, 0, 7, 1, 2, 3\}$$



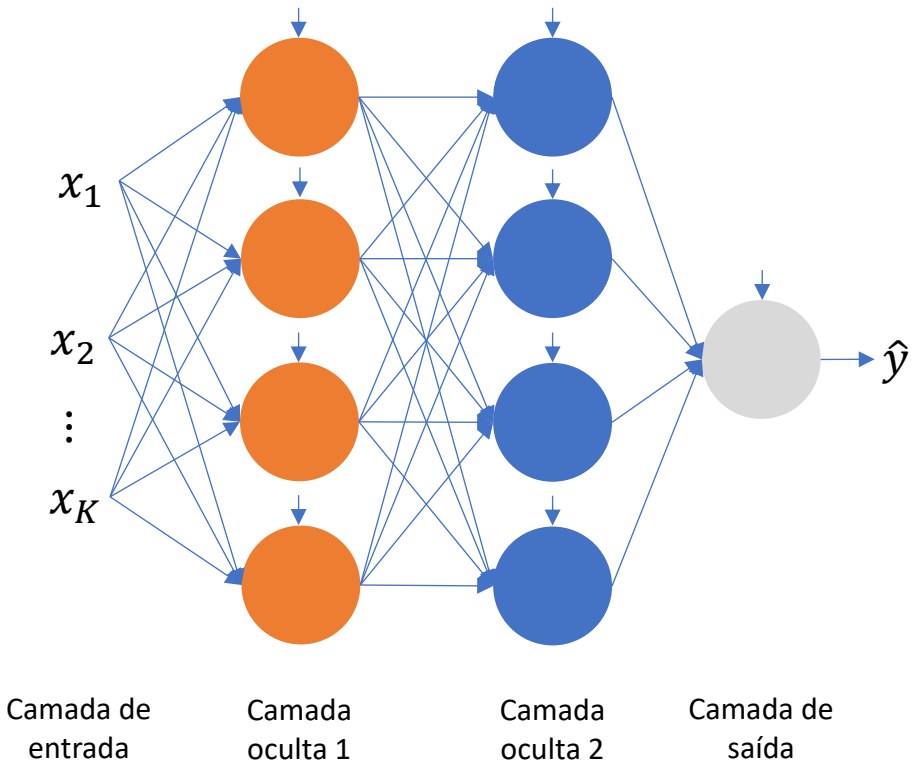
- Para aproximar esses dados, podemos usar a ***equação de um plano*** como nossa ***função hipótese***:

$$\hat{y} = a_0 + a_1 x_1 + a_2 x_2.$$

- Usando ***ativações lineares***, a rede neural abaixo representa a ***função do plano***.



Regressão

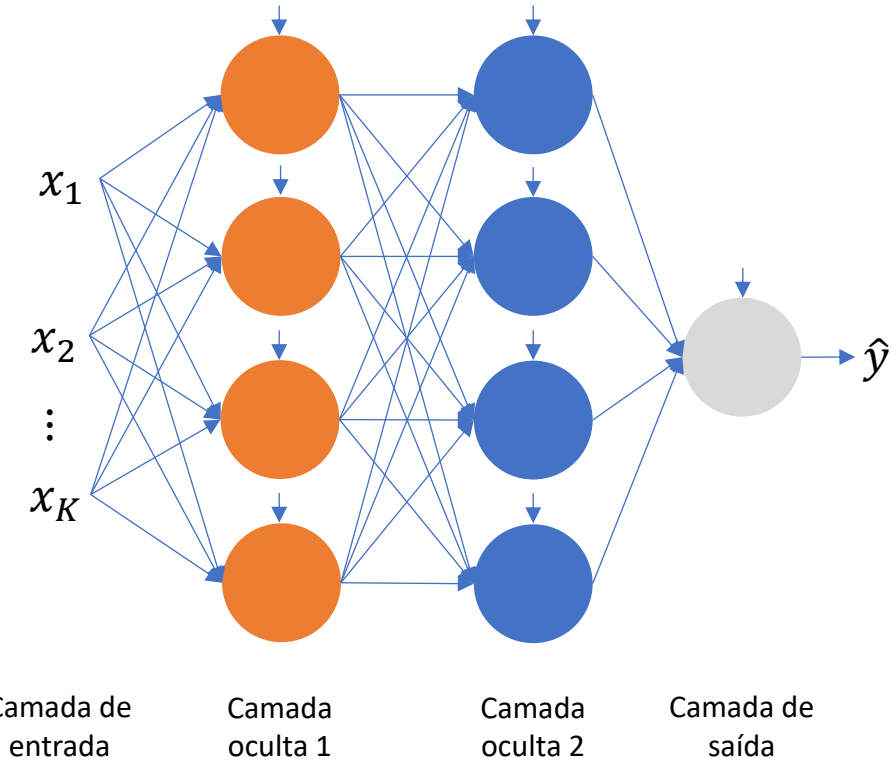


- Podemos extrapolar isso pra quantos ***atributos*** forem necessários.
- O modelo ao lado tem K ***atributos*** (i.e., K entradas).
- Com ***ativações lineares***, a rede neural ao lado representa a ***função de um hiperplano***

$$\hat{y} = a_0 + a_1 x_1 + a_2 x_2 + \cdots + a_K x_K$$
$$= \sum_{i=0}^{K} a_i x_i,$$

onde $x_0 = 1$.

Aproximação universal de funções



- Com ***ativações não lineares*** (sigmóide, *relu*, etc.) ***e uma camada oculta***, podemos ***aproximar qualquer tipo de função contínua***, incluindo o hiperplano, bastando encontrar o número de neurônios necessários.
- Com ***duas camadas ocultas***, podemos ***aproximar até funções com descontinuidades***.

Mas como encontramos o número ideal de camadas e neurônios?

Otimização hiperparamétrica

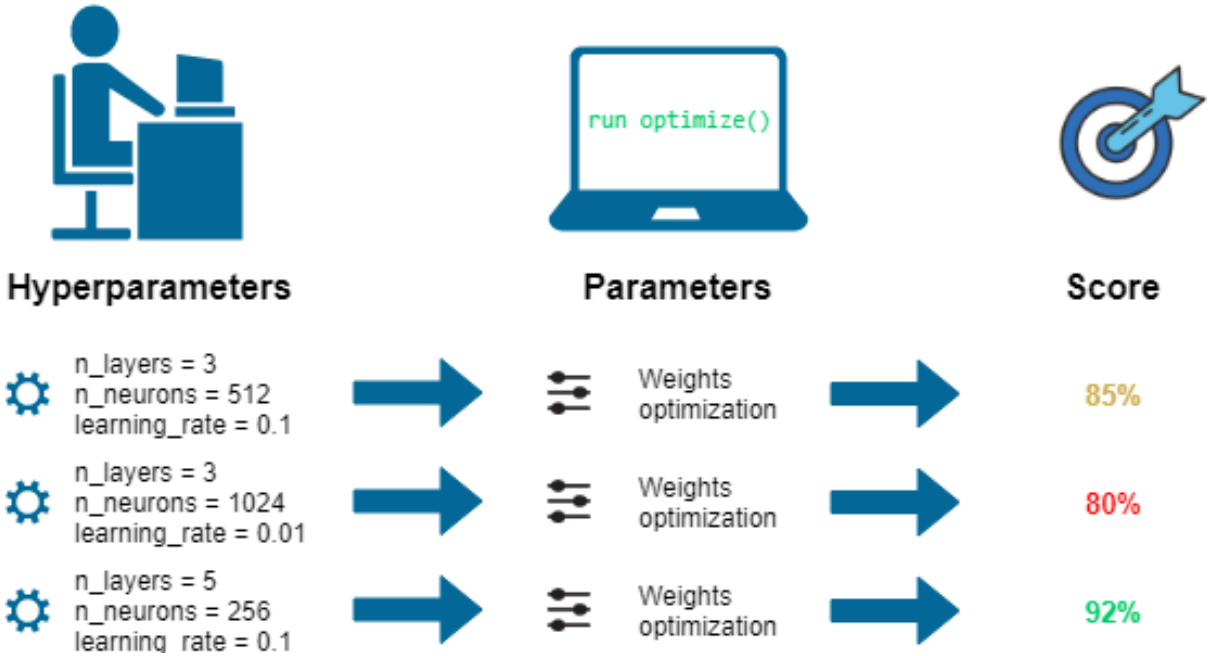
- É o processo de ***encontrar os melhores conjuntos de hiperparâmetros*** para um modelo de ML.
- Hiperparâmetros são ***parâmetros que não são aprendidos durante o treinamento*** do modelo, mas que afetam seu ***desempenho e comportamento***.





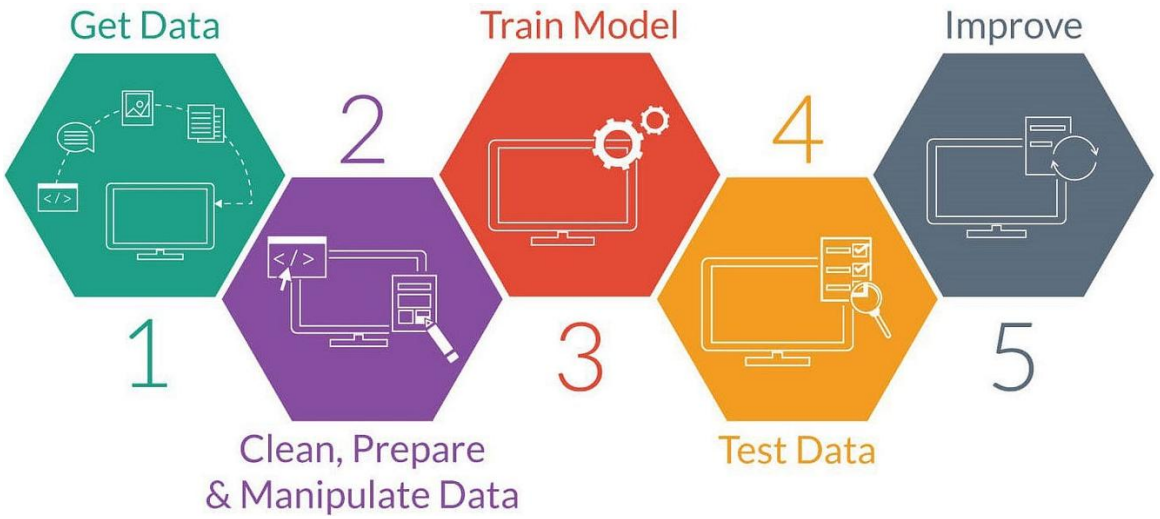


Otimização hiperparamétrica



- Exemplos de hiperparâmetros incluem a ***taxa de aprendizagem***, ***número de camadas*** e ***neurônios***, ***tamanho do mini-batch***, ***otimizador***, e muitos outros.
- Existem diversas bibliotecas que automatizam a busca pelos melhores hiperparâmetros.
- As mais populares são:
  - KerasTuner,
  - Optuna,
  - Scikit-learn,
  - Hyperopt, etc.

ML Workflow



- Antes de vermos um exemplo mais complexo sobre regressão com redes neurais densas, vamos falar sobre o fluxo de trabalho com modelos de ML.
- O fluxo de trabalho refere-se à ***sequência de etapas que um projeto de aprendizado de máquina segue***, desde a coleta de dados até a implantação do modelo em produção.

ML Workflow



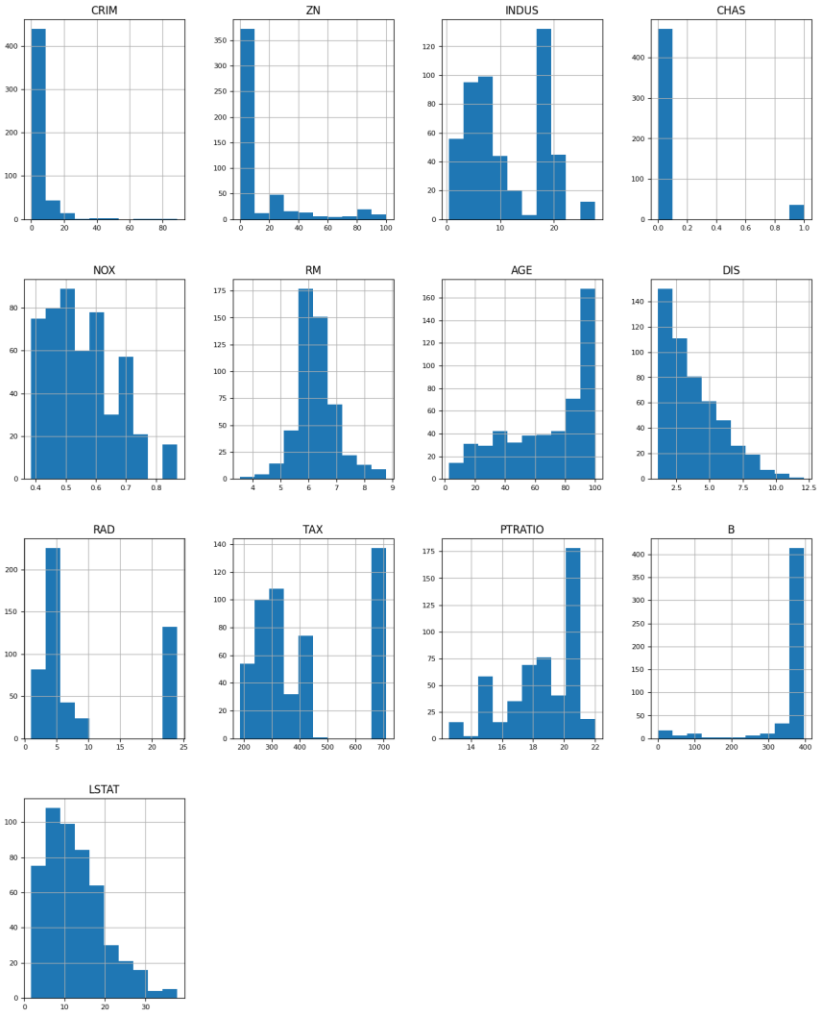

```
data = tf.keras.datasets.boston_housing

(x_train, y_train), (x_test, y_test) = data.load_data()
```

- O primeiro passo no ***fluxo de trabalho*** com modelos de ML envolve a ***coleta de dados***.
- Podemos ***coletar dados*** realmente, por exemplo, gravar sons ou vídeos, tirar fotos, etc. ou ***reusar um conjunto de dados existente***.

ML Workflow





- Na sequência, fazemos uma ***análise exploratória dos dados*** (*exploratory data analysis* – EDA), avaliando intervalos, tipos e correlação entre os atributos e buscando valores faltantes, duplicados, discrepantes, etc.

ML Workflow




```
from sklearn.preprocessing import StandardScaler

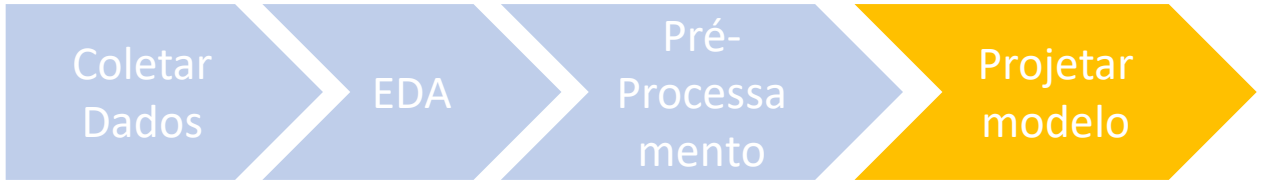
scaler = StandardScaler()

scaler.fit(x_train)

x_train_std = scaler.transform(x_train)
x_test_std = scaler.transform(x_test)
```

- Em seguida, realizamos o ***pré-processamento*** dos dados.
- Essa tarefa pode envolver a ***remoção de valores discrepantes***, transformação de atributos (e.g., categóricos nominais em quantitativos), ***preenchimento ou remoção de exemplos*** com dados incompletos ou duplicados, ***escalonamento dos atributos***, ***data augmentation***, etc.

ML Workflow




```
model = tf.keras.models.Sequential(
    [
        tf.keras.layers.Dense(20, input_shape=[13], activation='relu'),
        tf.keras.layers.Dense(1)
    ]
)

model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

- Após, temos a fase de ***criação do modelo***.
- Envolve a ***definição da arquitetura***: quantidade de camadas, número de nós por camada, funções de ativação, otimizador, passo de aprendizagem, métricas, etc.

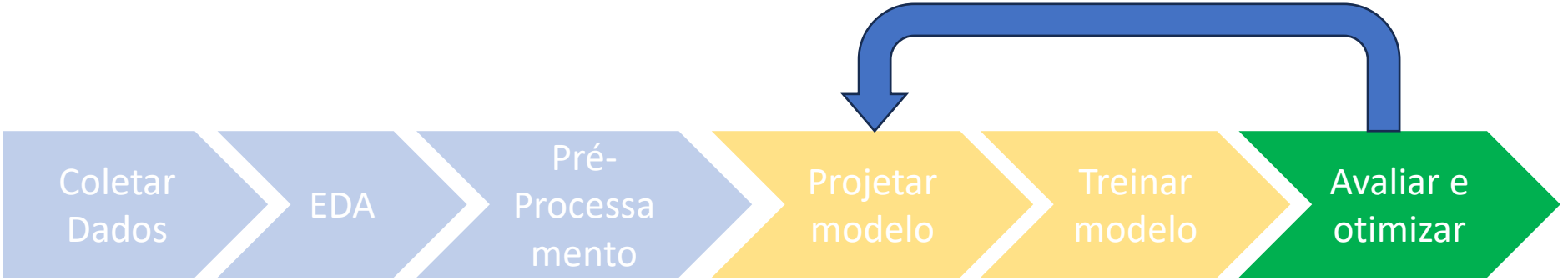
ML Workflow

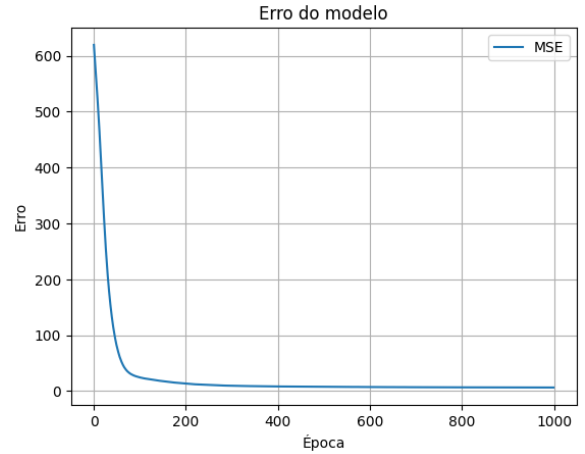



```
history = model.fit(
    x_train_std,
    y_train,
    epochs=1000,
)
```

- O ***treinamento*** do modelo vem na sequência.
- A entrada desta etapa são os dados já pré-processados.

ML Workflow






```
test_eval = model.evaluate(x_test_std, y_test)
train_eval = model.evaluate(x_train_std, y_train)

tuner.search(
   x_train_std, y_train,
   epochs=500,
   validation_data=(x_test_std, y_test)
)
```

- ***Análise*** dos resultados obtidos após o treinamento.
- Analisar indícios de que o modelo está aprendendo:
  - Erros de treinamento e validação devem diminuir ao longo do treinamento até que se tornem praticamente constantes (indicação de convergência).
  - Ambos os erros devem ser pequenos e próximos.
- Se o modelo não estiver aprendendo, devemos ***otimizá-lo manualmente ou através de técnicas de otimização hiperparamétrica***.

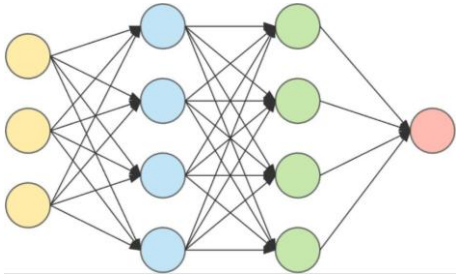
ML Workflow




```
xt = np.array([1.1, 0., 9., 0., 0.6, 7., 92., 3.8 , 4., 300., 21., 200, 19.5])
xt = np.reshape(xt, (1, 13))
xt_norm = scaler.transform(xt)
yt = model.predict(xt_norm)
```

- Após obtermos um bom modelo, o colocamos em "***produção***" para lidar com dados do mundo real (i.e., inéditos) e oferecer *insights* ou auxiliar em tomadas de decisão.

ML Workflow







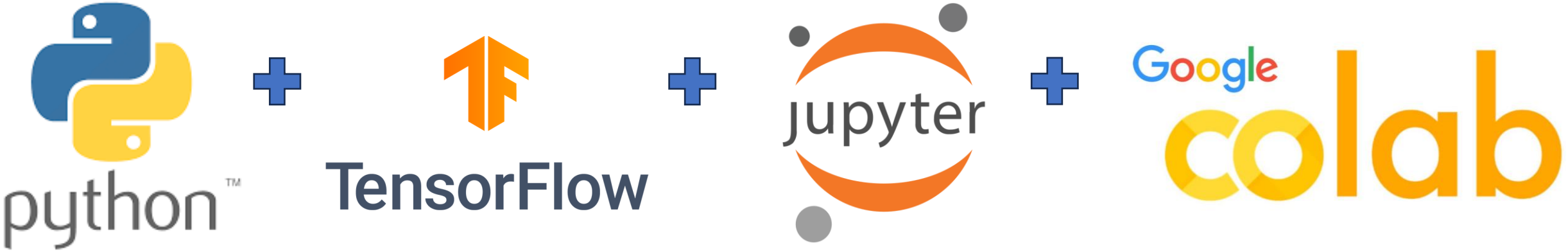




- Esse é o fluxo de trabalho que geralmente seguimos para trabalhar com modelos de aprendizado de máquina.
- O ***fluxo com o EdgeML terá uma fase adicional intermediária entre avaliar/otimizar e a inferência***, que será a ***etapa de conversão*** (e.g., compressão) do modelo para o executarmos em dispositivos embarcados.

Exemplo

- Regressão de preços de residências usando redes neurais densas (DNNs)



Atividades

- Exercício #1: Regressão sem escalonamento
- Exercício #2: Otimização hiperparamétrica

Obrigado!